

Tugas Modul Praktikum 3

Mata Kuliah Pengembangan Aplikasi Mobile

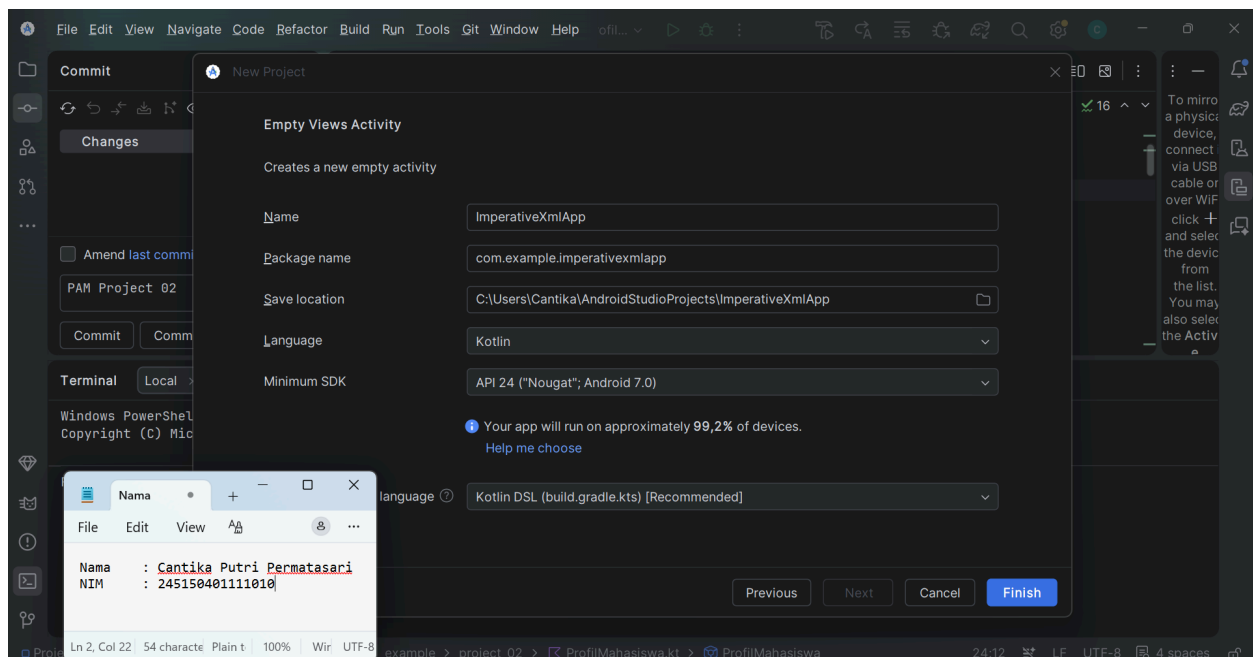
Nama : Cantika Putri Permatasari

NIM : 245150401111010

Materi : *Event, State, dan Implisit Intent* pada *Jetpack Compose*

Project A - XML/View (Imperatif)

1. Buat project Android Studio baru menggunakan template Empty Views Activity. Beri nama project ImperativeXmlApp dan pilih Kotlin.



2. Buka res/layout/activity_main.xml, kemudian gunakan kode berikut:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="24dp">

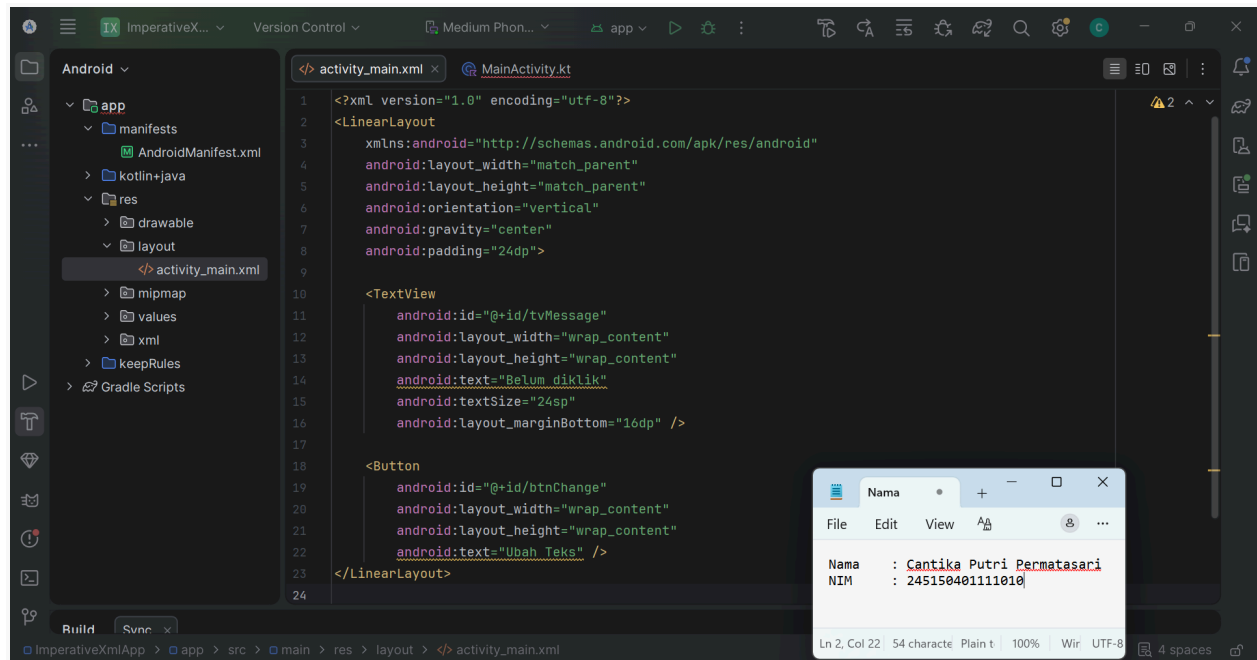
    <TextView
        android:id="@+id/tvMessage"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Belum diklik">
```

```

        android:textSize="24sp"
        android:layout_marginBottom="16dp" />

<Button
    android:id="@+id/btnChange"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Ubah Teks" />
</LinearLayout>

```



3. Buka MainActivity.kt. Sesuaikan package dengan package project, kemudian gunakan kode berikut:

```

package com.example.imperativexmlapp

import android.os.Bundle
import android.widget.Button
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity

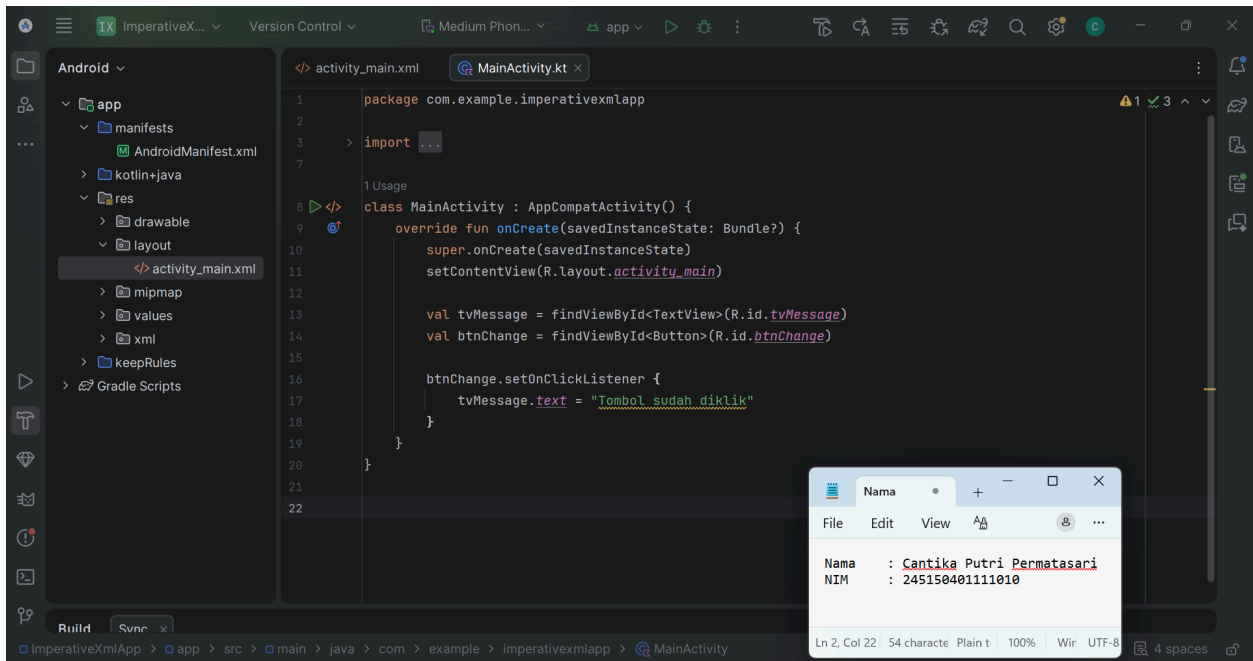
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val tvMessage = findViewById<TextView>(R.id.tvMessage)
        val btnChange = findViewById<Button>(R.id.btnChange)

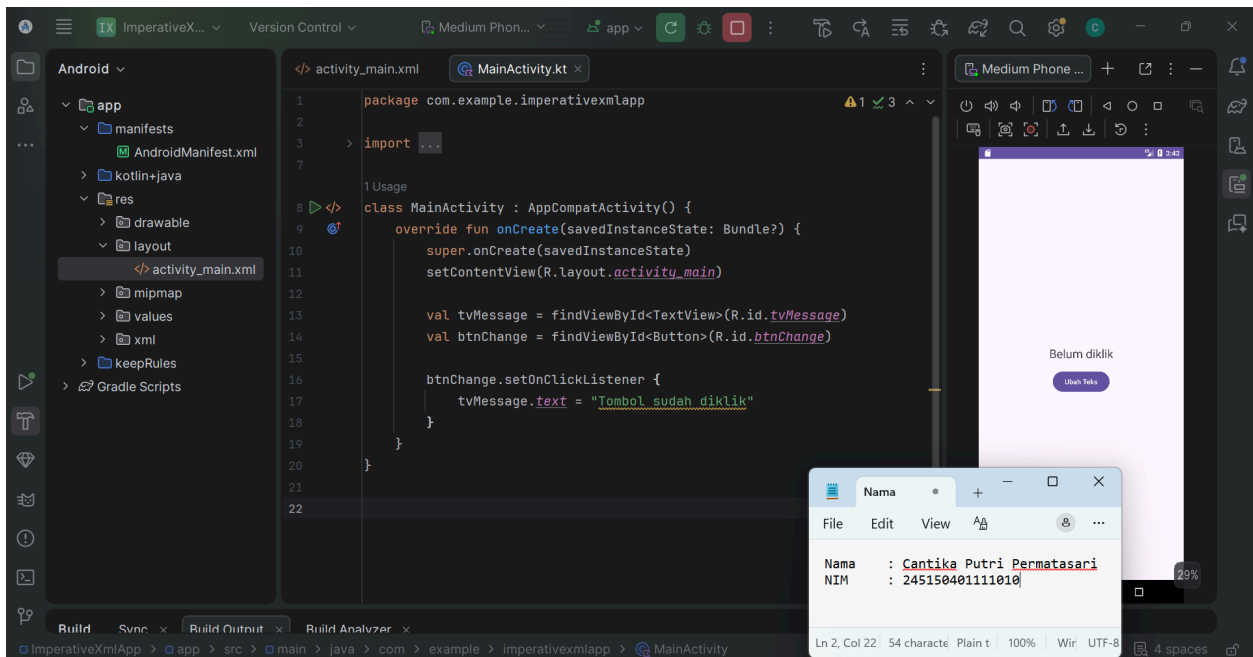
        btnChange.setOnClickListener {
            tvMessage.text = "Tombol sudah diklik"
        }
    }
}

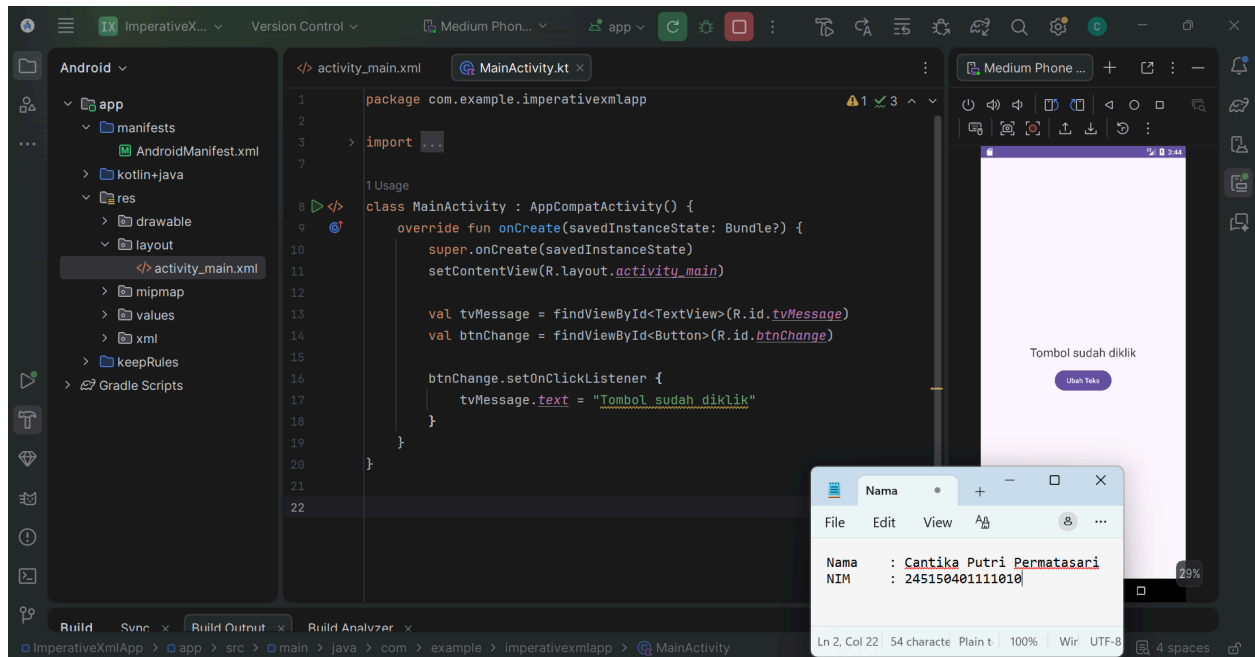
```

```
}  
}
```



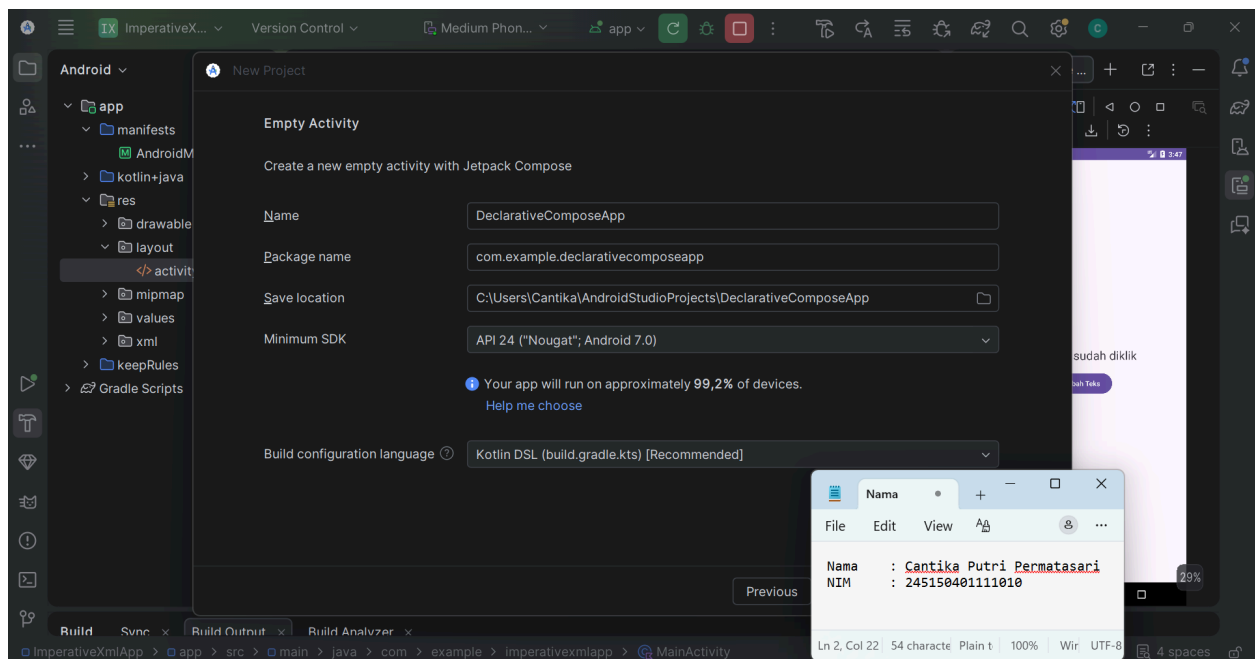
4. Jalankan aplikasi. Pastikan teks berubah setelah tombol diklik.





Project B - Jetpack Compose (Deklaratif)

1. Buat project Android Studio baru menggunakan template Empty Activity untuk Jetpack Compose. Beri nama project DeclarativeComposeApp dan pilih Kotlin.



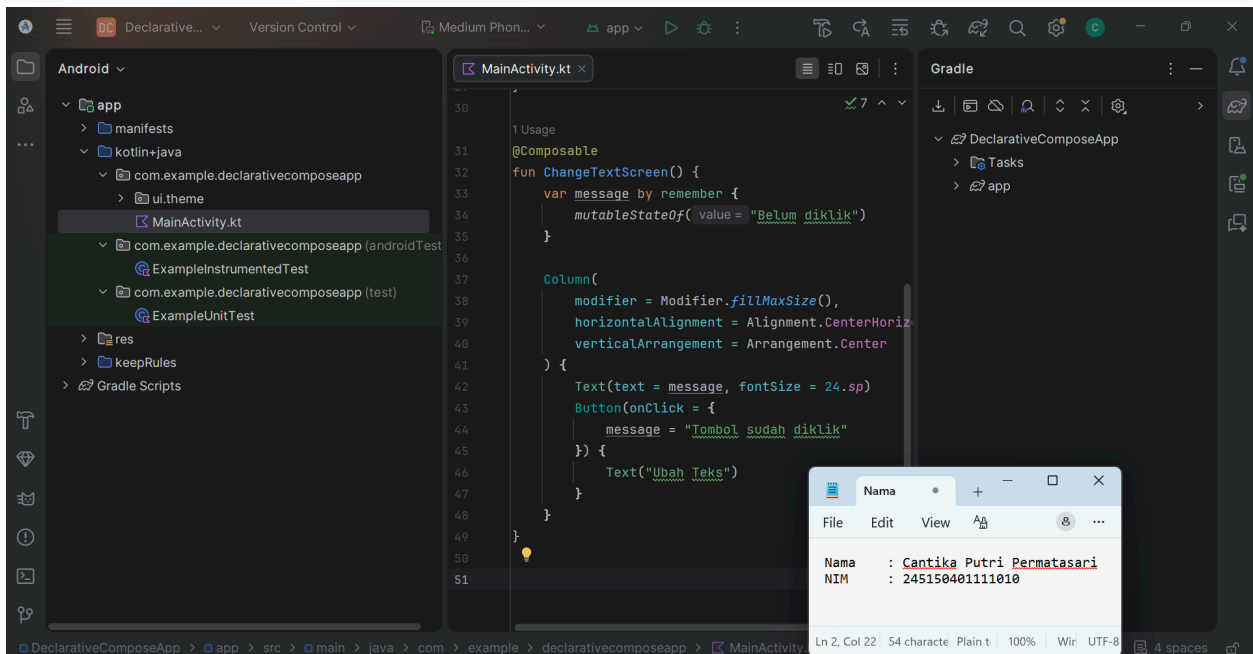
2. Buka MainActivity.kt, pertahankan package project, kemudian gunakan fungsi composable berikut:

```

@Composable
fun ChangeTextScreen() {
    var message by remember {
        mutableStateOf("Belum diklik")
    }

    Column(
        modifier = Modifier.fillMaxSize(),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Text(text = message, fontSize = 24.sp)
        Button(onClick = {
            message = "Tombol sudah diklik"
        }) {
            Text("Ubah Teks")
        }
    }
}

```



3. Tambahkan import yang diperlukan:

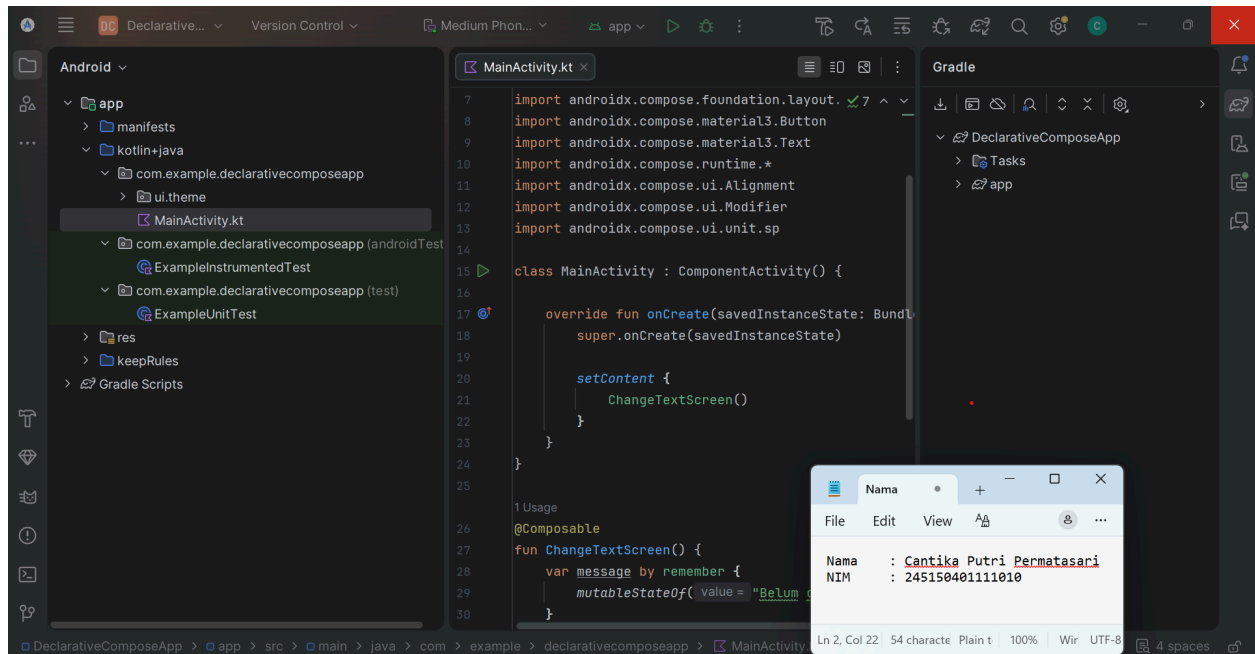
```

import androidx.compose.foundation.layout.*
import androidx.compose.material3.Button
import androidx.compose.material3.Text
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.sp

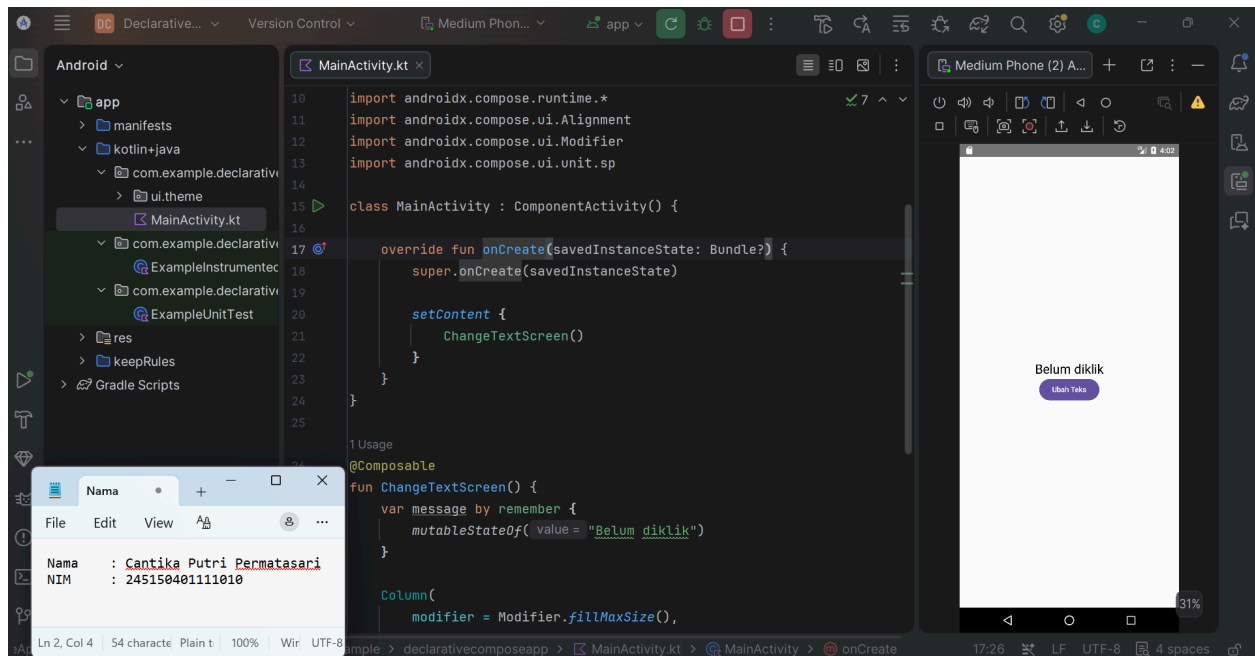
```

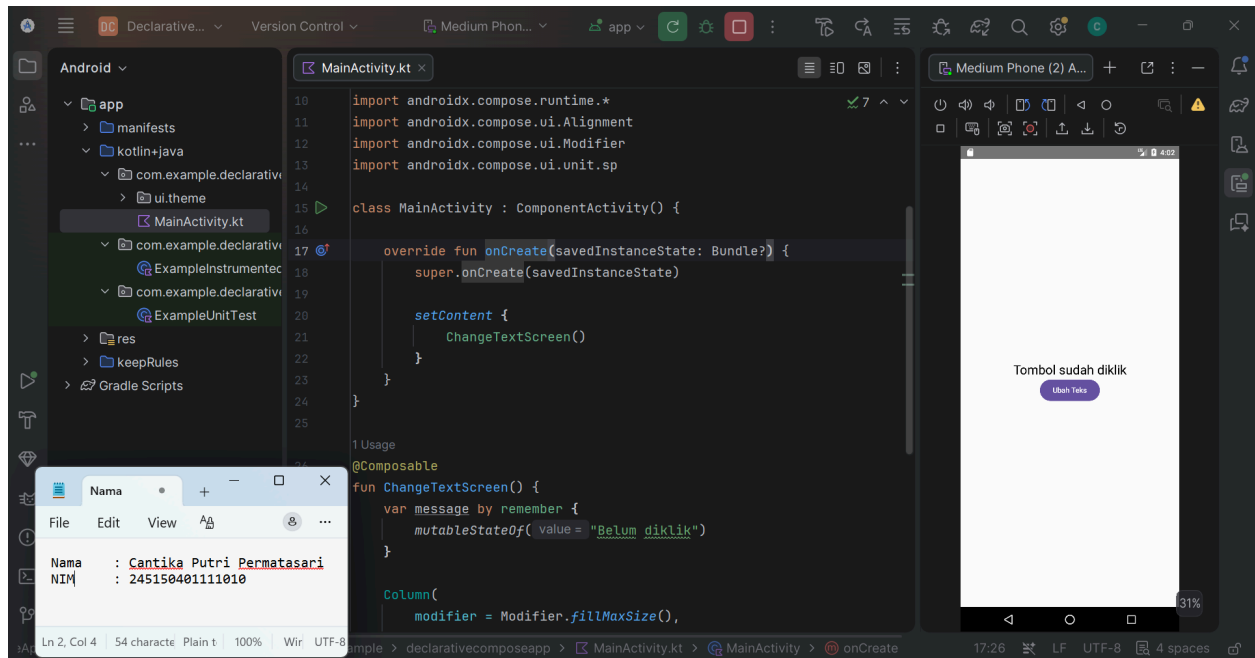
4. Pada fungsi onCreate(), ubah isi setContent agar memanggil ChangeTextScreen():

```
setContent {  
    ChangeTextScreen()  
}
```



5. Jalankan aplikasi. Pastikan hasil tampilannya sama dengan project XML/View.

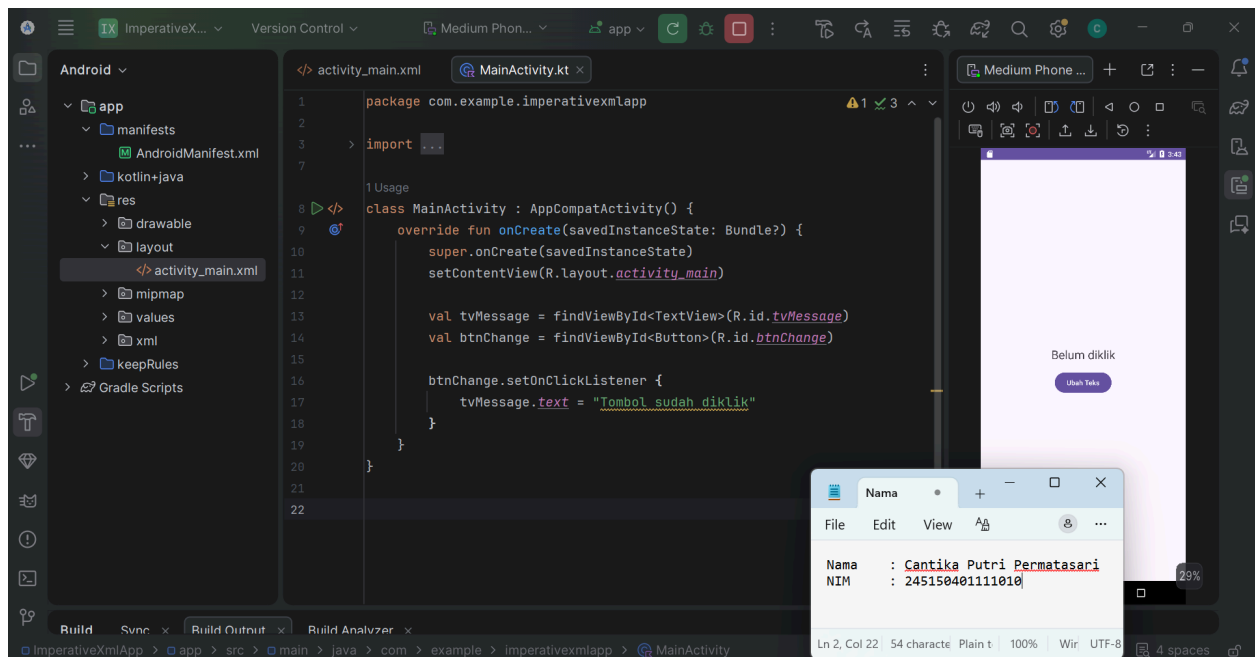


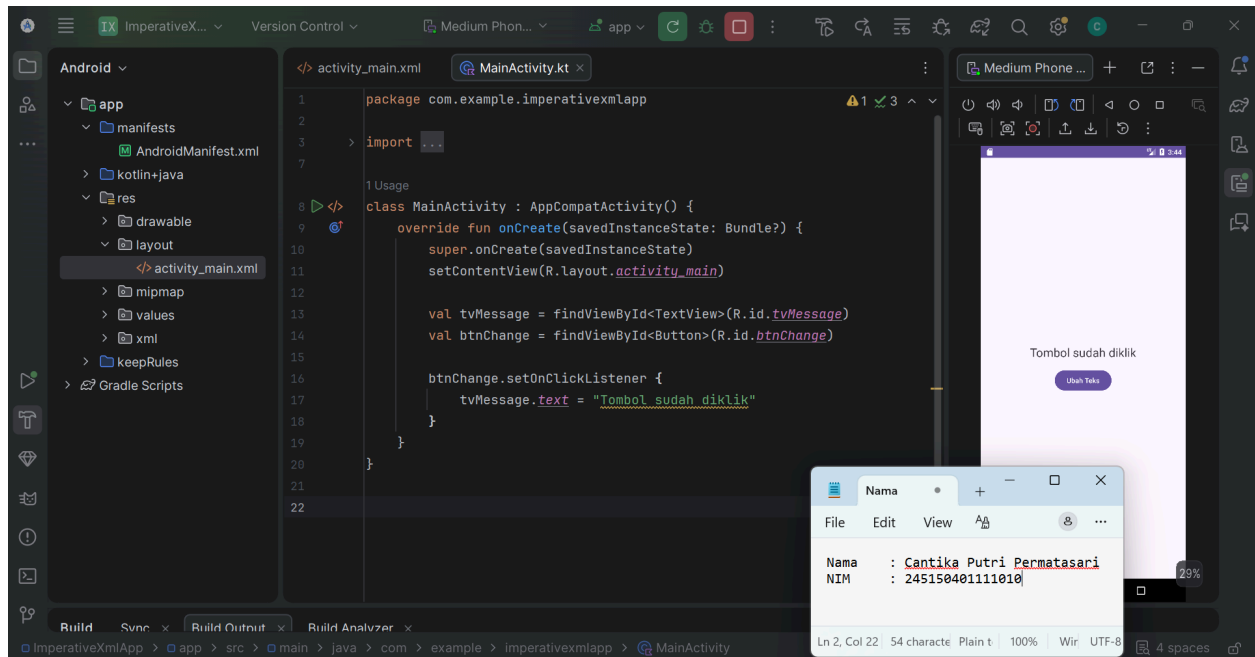


Aktivitas Percobaan

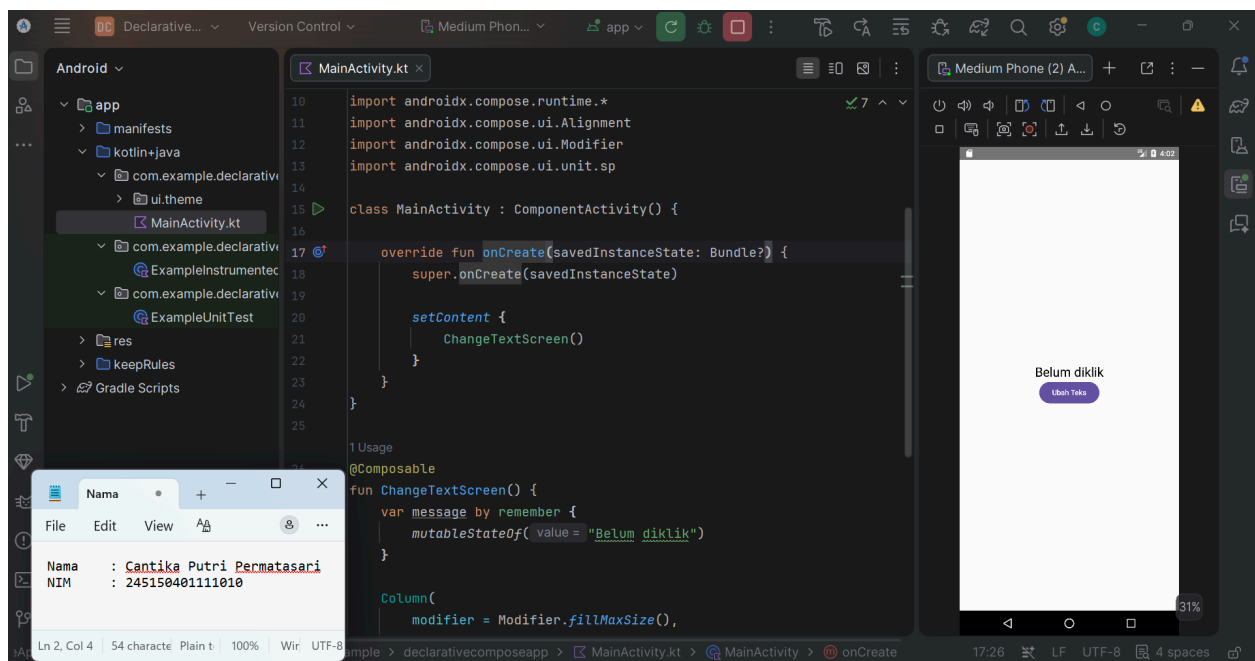
1. Jalankan kedua aplikasi dan ambil screenshot kondisi sebelum serta sesudah tombol diklik.

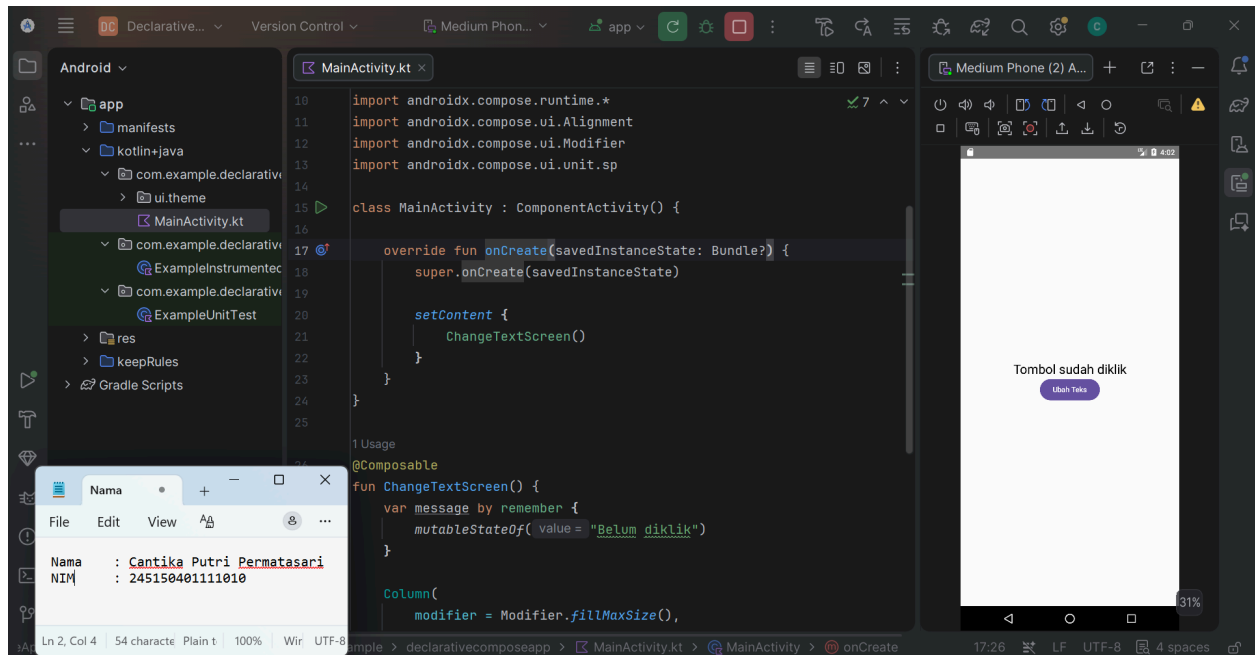
Project A - XML/View (Imperatif)





Project B - Jetpack Compose (Deklaratif)





2. Pada project XML/View, objek apa yang diubah secara langsung ketika tombol diklik? Tunjukkan baris kodenya.

Pada project XML/View objek yang diubah secara langsung adalah `TextView` dengan id `tvMessage`. Perubahan dilakukan pada properti `text` milik `TextView`

Baris kodenya:

```
btnChange.setOnClickListener {
    tvMessage.text = "Tombol sudah diklik"
}
```

Jadi, ketika tombol diklik, program langsung mengubah tampilan `TextView` dari `"Belum diklik"` menjadi `"Tombol sudah diklik"`

3. Pada project Compose, nilai apa yang diubah ketika tombol diklik? Tunjukkan baris kodenya.

Pada Compose, yang diubah adalah nilai state message, bukan objek Text secara langsung.

Baris kodenya:

```
Button(onClick = {
    message = "Tombol sudah diklik"
})
```

4. Mengapa pada Compose tidak terdapat perintah seperti `tvMessage.text` atau `setText()`?

Karena Compose menggunakan pendekatan deklaratif. Komponen tampilan tidak perlu diubah secara langsung. Cukup menentukan bagaimana tampilan seharusnya berdasarkan nilai state, sehingga ketika state berubah, tampilan akan diperbarui secara otomatis.

5. Jelaskan alur perubahan pada XML/View dengan pola: event -> View -> perubahan tampilan.

Pada XML/View, perubahan dimulai ketika terjadi event berupa tombol yang diklik. Event tersebut menjalankan OnClickListener, kemudian mengakses View `tvMessage` dan mengubah nilai text secara langsung. Setelah nilai text diubah menjadi "Tombol sudah diklik", tampilan TextView pada layar ikut berubah.

6. Jelaskan alur perubahan pada Compose dengan pola: event -> state -> recomposition -> perubahan tampilan.

Pada Compose, perubahan dimulai ketika terjadi event berupa tombol yang diklik. Event tersebut mengubah state message menjadi "Tombol sudah diklik". Perubahan state kemudian memicu recomposition, sehingga Compose menjalankan kembali bagian tampilan yang bergantung pada state tersebut. Setelah itu, tampilan Text diperbarui dan menampilkan teks "Tombol sudah diklik".

7. Apa yang terjadi jika message diganti menjadi variabel biasa: `var message = "Belum diklik"`? Jelaskan penyebabnya.

Jika message diganti menjadi variabel biasa, perubahan nilai message tidak akan memperbarui tampilan secara otomatis. Hal ini karena variabel biasa bukan merupakan state yang dapat diamati oleh Compose, sehingga perubahan nilainya tidak memicu recomposition.

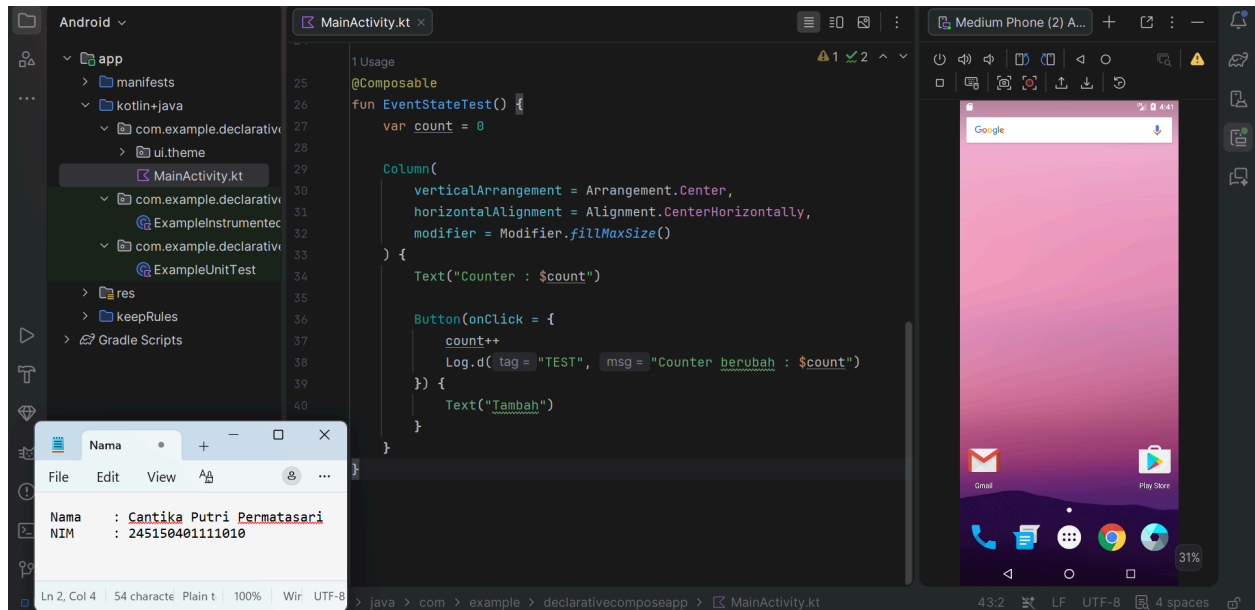
8. Tuliskan kesimpulan perbedaan cara berpikir imperatif dan deklaratif menggunakan kalimat Anda sendiri.

Pendekatan imperatif berfokus pada langkah yang harus dilakukan untuk mengubah tampilan secara langsung, sedangkan pendekatan deklaratif berfokus pada kondisi tampilan berdasarkan state. Pada Compose, perubahan state akan memicu recomposition sehingga tampilan diperbarui secara otomatis.

Event Handling State

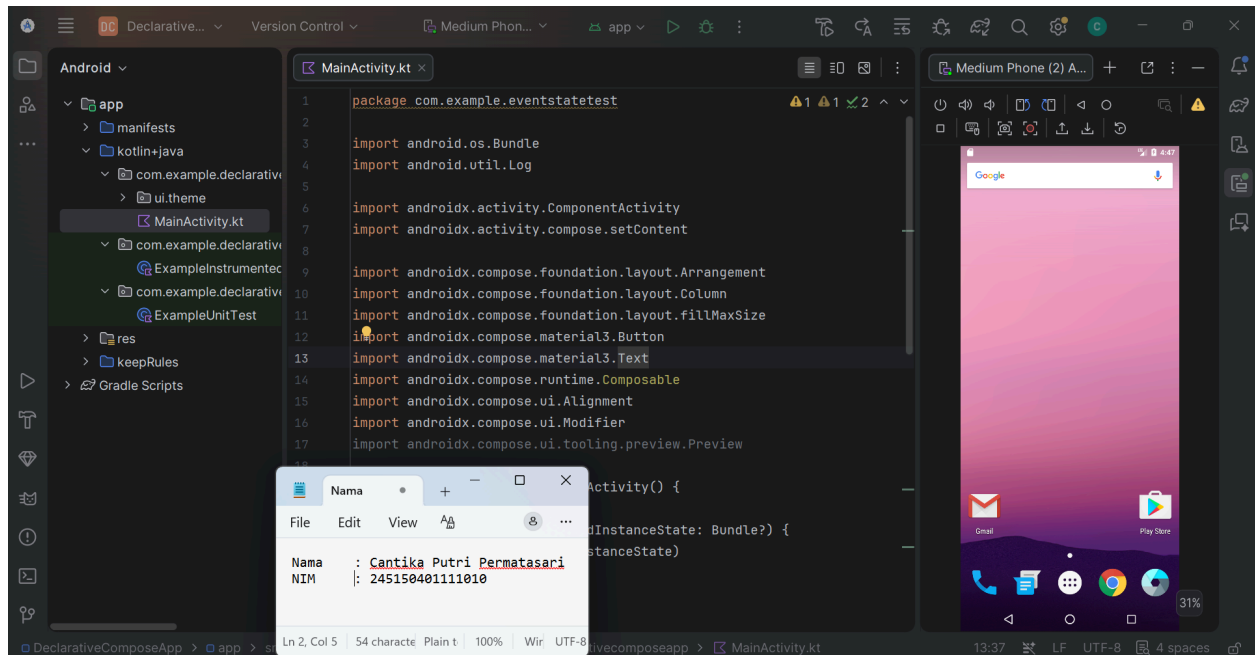
1. Buat sebuah Project Android Studio baru, dengan Template Empty Activity untuk Phone and Tablet. Pastikan project yang dibuat menggunakan Jetpack Compose. Tunggu sampai proses Build inisialisasi project baru ini selesai dilakukan sebelum melanjutkan percobaan ke langkah selanjutnya.

2. Buka file `MainActivity.kt` dan tambahkan fungsi composable berikut ini di luar class `MainActivity`:



3. Import semua keyword dan konstruk atas kesalahan sintaksis dan struktur data yang muncul sebagai akibat dari penambahan kode program yang baru ditambahkan tersebut, termasuk namun tidak terbatas pada:

```
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.material3.Button
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.tooling.preview.Preview
```



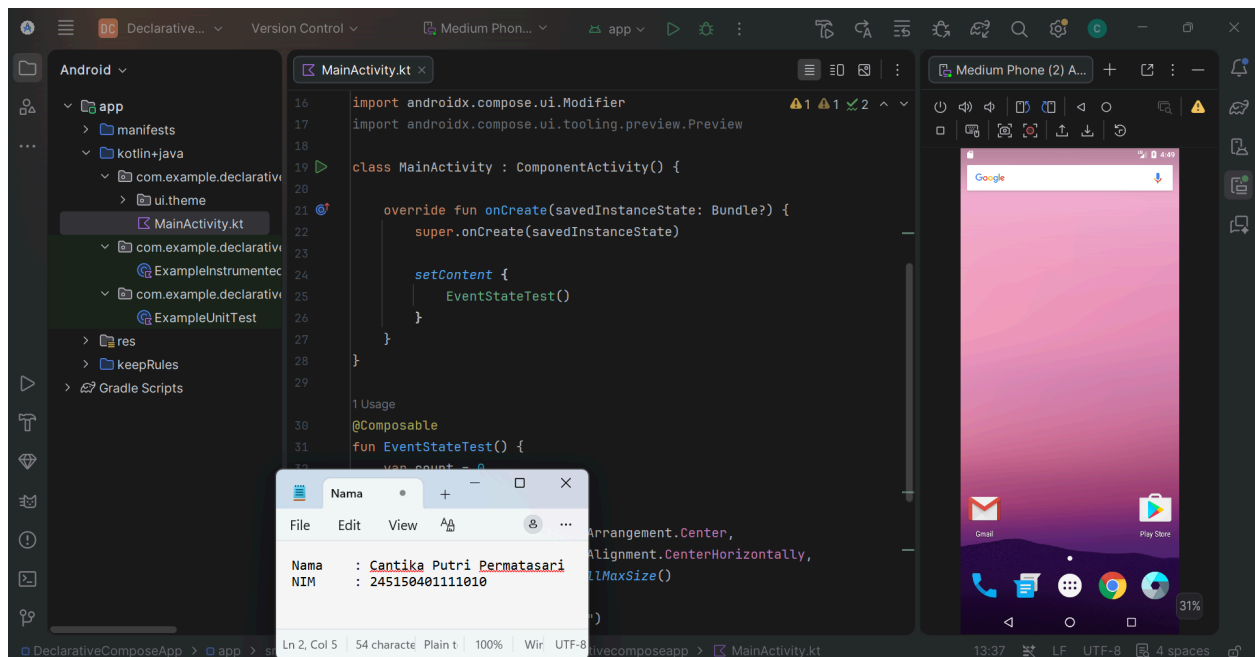
4. Buka file `MainActivity.kt`, temukan fungsi `onCreate()` yang telah disediakan oleh template aplikasi Android di Android Studio. Ubah isi `setContent {}` menjadi

```

setContent {
    EventStateTest()
}

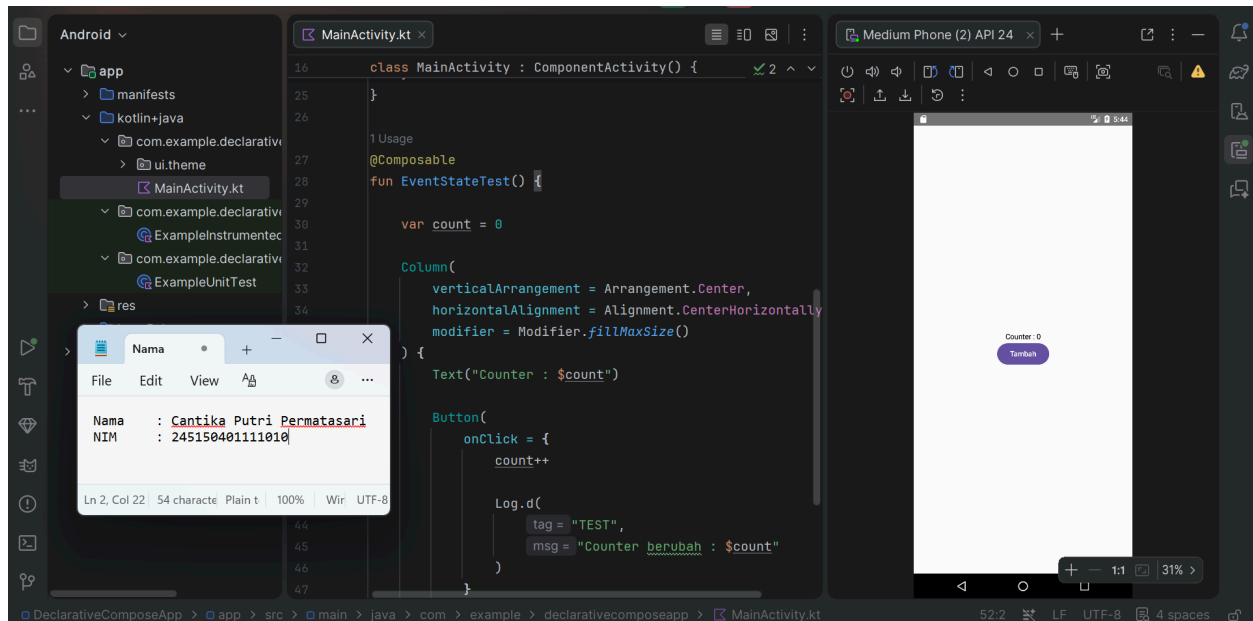
```

5. Sebelum menjalankan aktivitas percobaan, pastikan bahwa tidak ada penulisan kode program yang salah yang dapat mengakibatkan program tidak dapat dijalankan dengan baik pada saat di-compile.



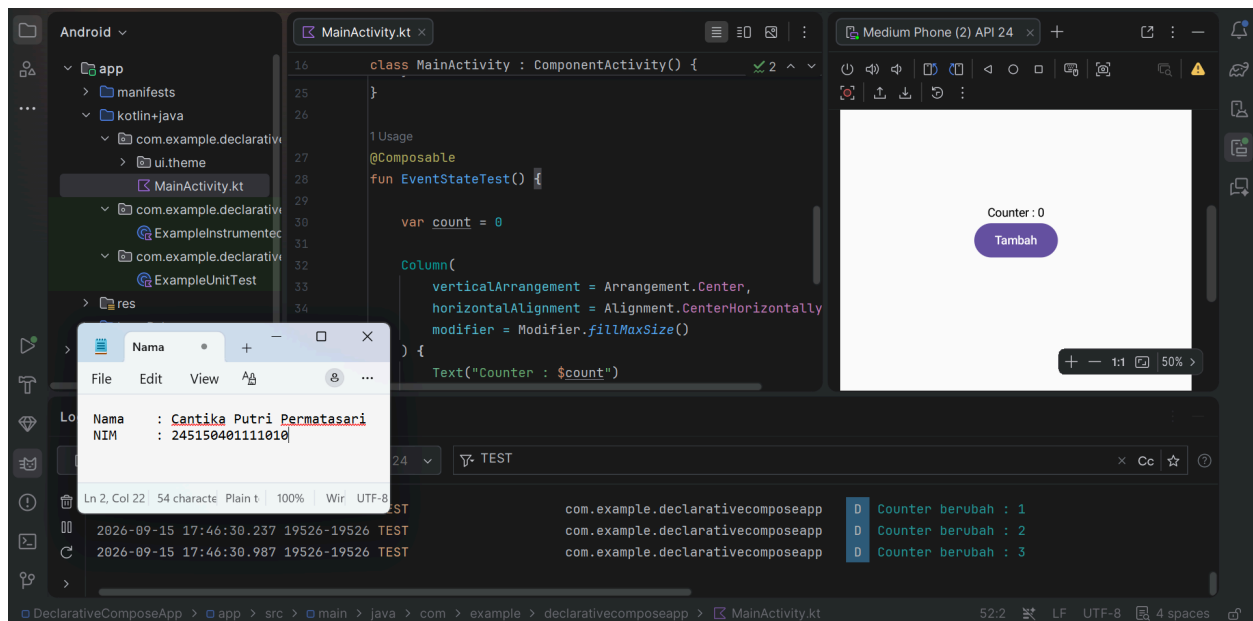
Aktivitas Percobaan

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.



2. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?

Setiap tombol Tambah diklik, LogCat menampilkan pesan seperti berikut:



Angka akan bertambah setiap kali tombol ditekan.

3. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?

Pada percobaan pertama, angka Counter pada tampilan tidak berubah dan tetap menunjukkan Counter : 0 meskipun nilai count sebenarnya bertambah ketika tombol diklik.

4. Apa yang menyebabkan hal tersebut terjadi?

Hal tersebut terjadi karena count merupakan variabel biasa, bukan state Compose. Perubahan nilai count tidak terdeteksi oleh Compose sehingga tidak memicu recomposition dan tampilan tidak diperbarui.

5. Langkah selanjutnya adalah mengubah kode sebelumnya untuk menggunakan state yaitu pada Composable EventStateTest.

6. Ubah kode program pada bagian fungsi Composable EventStateTest dari yang sebelumnya:

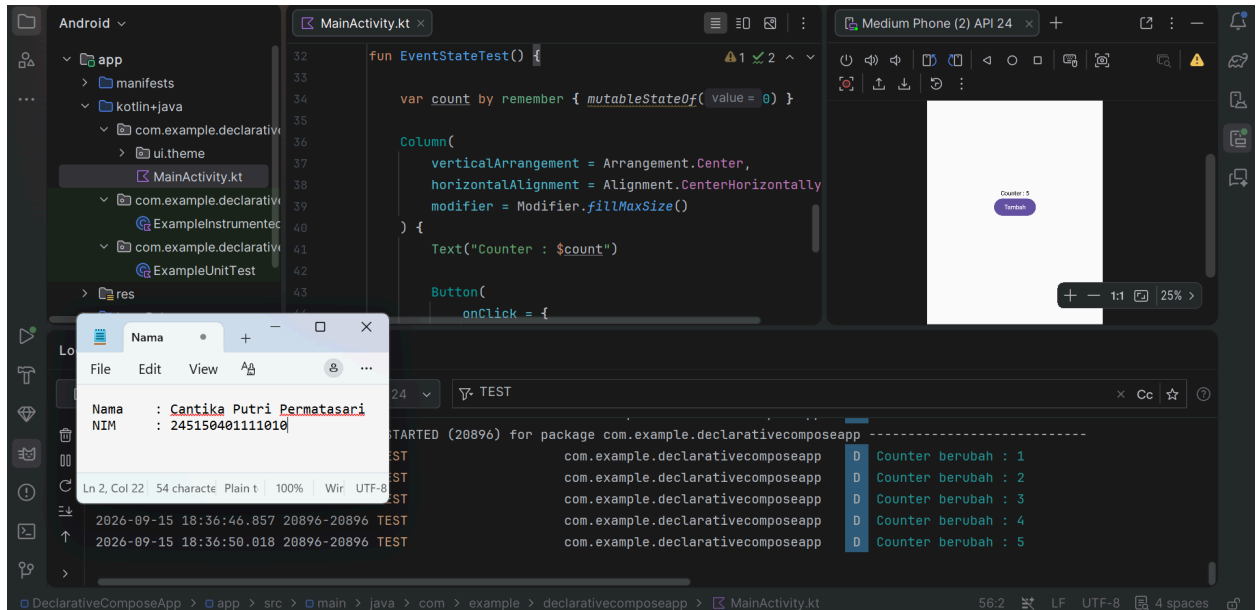
```
var count = 0
```

menjadi:

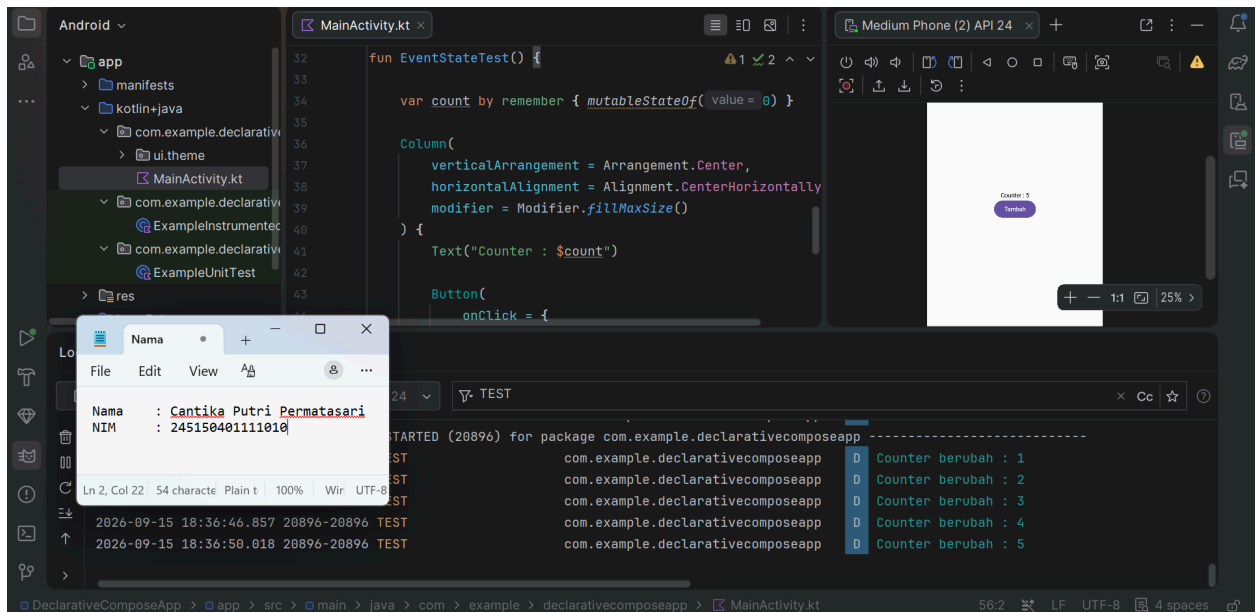
```
var count by remember { mutableStateOf(0) }
```

7. Berikut kode lengkap EventStateTest:

```
@Composable
fun EventStateTest() {
    var count by remember { mutableStateOf(0) }
    Column(
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally,
        modifier = Modifier.fillMaxSize()
    ) {
        Text("Counter : $count")
        Button(onClick = {
            count++
            Log.d("TEST", "Counter berubah : $count")
        }) {
            Text("Tambah")
        }
    }
}
```



8. Jalankan program kembali, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.



9. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?
Setiap kali tombol Tambah diklik, LogCat menampilkan pesan perubahan nilai count.
10. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?
Tampilan angka yang ada pada teks "Counter : 0" di layar kini ikut berubah dan bertambah (menjadi 1, 2, 3, 4, 5, dst) setiap kali tombol diklik.

11. Apa yang menyebabkan hal tersebut terjadi dan berbeda dengan percobaan sebelumnya?

Hal tersebut terjadi karena `count` sekarang menggunakan state dengan `remember { mutableStateOf(0) }`. Ketika nilai `count` berubah, Compose mendeteksi perubahan state dan melakukan recomposition sehingga tampilan ikut diperbarui. Pada percobaan sebelumnya, `count` hanya merupakan variabel biasa sehingga perubahan nilainya tidak menyebabkan tampilan diperbarui.

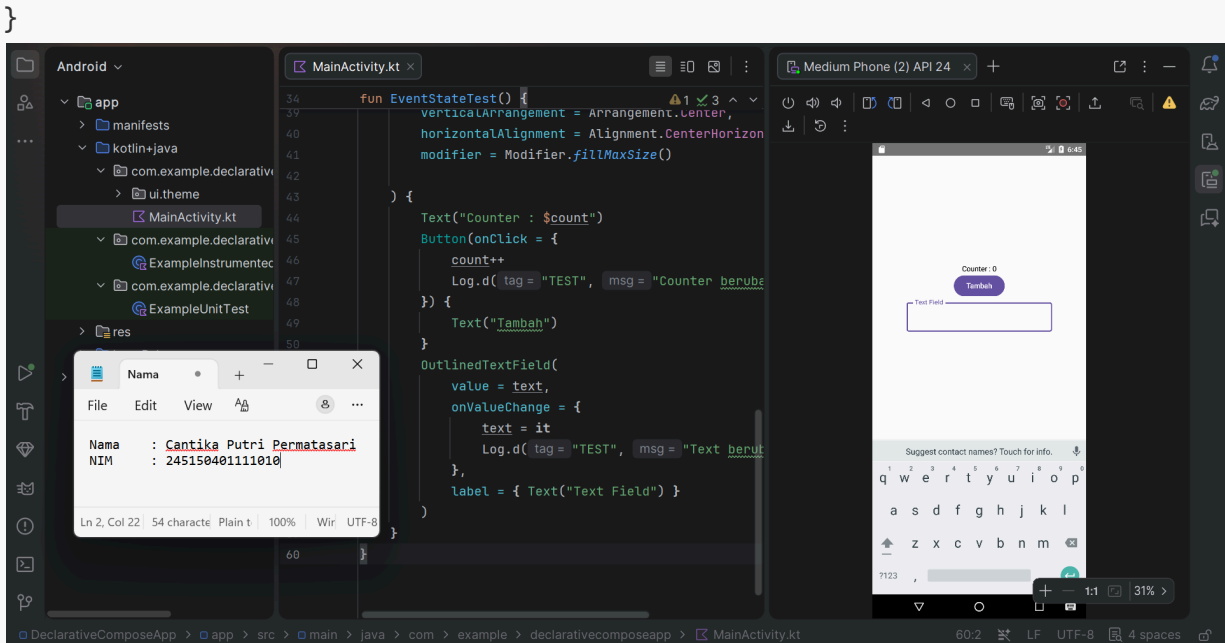
12. Apa yang dimaksud recomposition pada Compose?

Recomposition adalah proses ketika Compose menjalankan kembali bagian UI yang bergantung pada state yang berubah, sehingga tampilan dapat diperbarui sesuai dengan nilai state terbaru.

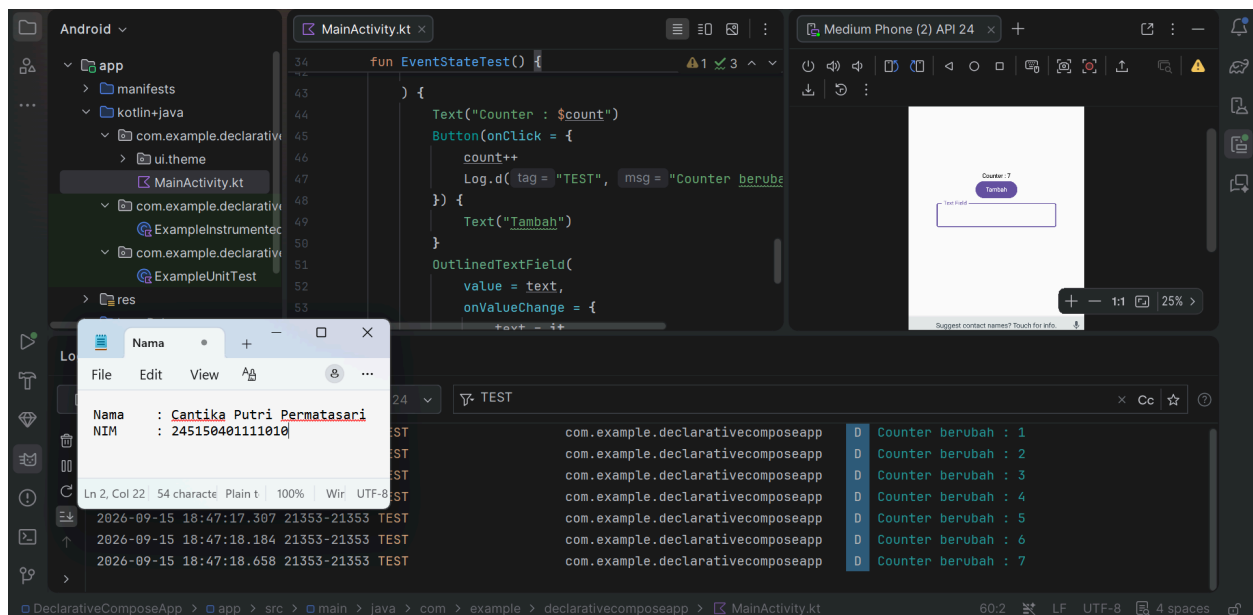
13. Langkah selanjutnya adalah menambah kode sebelumnya dengan menambah state baru yaitu text bertipe string pada Composable EventStateTest. Kemudian di dalam Column panggil composable OutlinedTextField di bawah Button Tambah. Penambahan kode program membuat kode pada fungsi Composable EventStateTest menjadi:

```
fun EventStateTest() {
    var count by remember { mutableStateOf(0) }
    var text by remember { mutableStateOf("") }

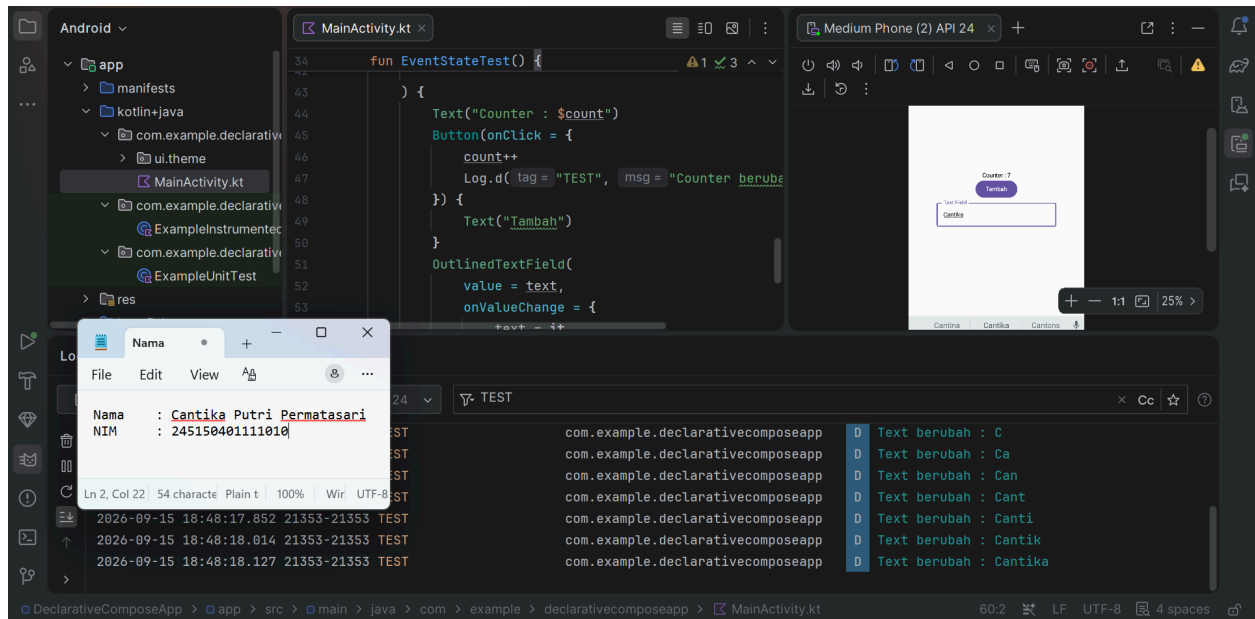
    Column(
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally,
        modifier = Modifier.fillMaxSize()
    ) {
        Text("Counter : $count")
        Button(onClick = {
            count++
            Log.d("TEST", "Counter berubah : $count")
        }) {
            Text("Tambah")
        }
        OutlinedTextField(
            value = text,
            onChange = {
                text = it
                Log.d("TEST", "Text berubah : $it")
            },
            label = { Text("Text Field") }
        )
    }
}
```



14. Jalankan program kembali, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.

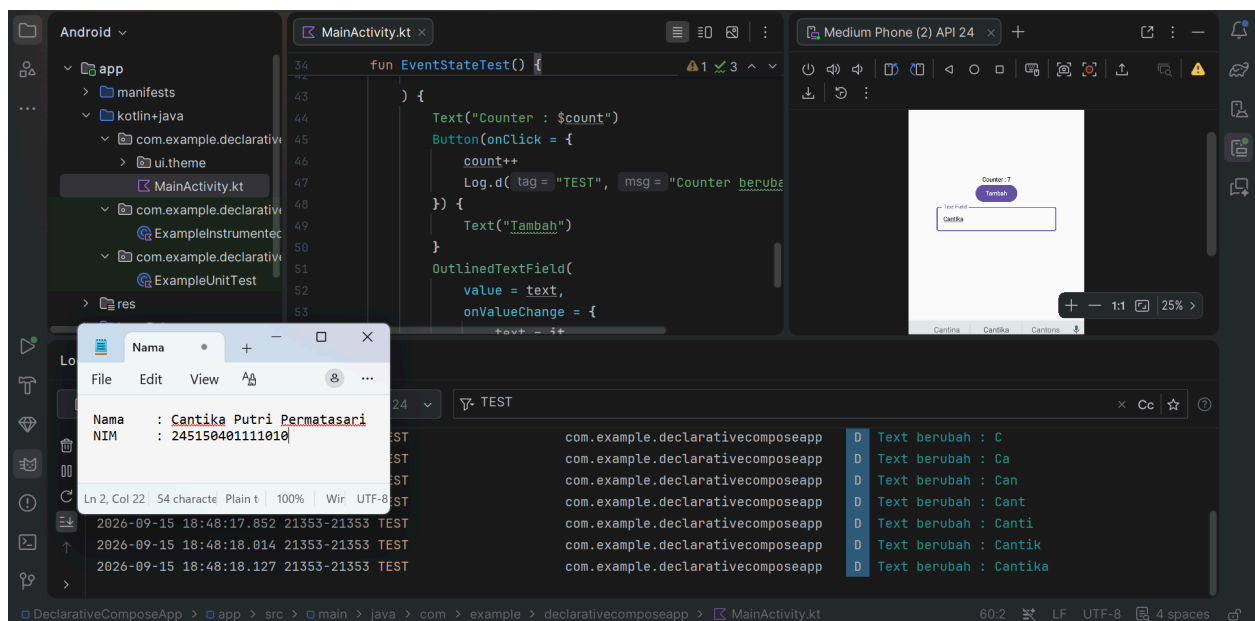


15. Apa yang muncul di LogCat setiap kali menginputkan huruf di Text Field?



Setiap huruf yang diketik langsung muncul di dalam Text Field dan isi Text Field terus diperbarui sesuai input.

16. Apa yang terjadi pada tampilan aplikasi setiap kali menginputkan huruf di Text Field?



Setiap huruf yang diinputkan akan langsung muncul pada Text Field, sehingga isi Text Field berubah sesuai dengan teks yang diketik.

17. Apa yang menyebabkan hal tersebut terjadi?

Hal tersebut terjadi karena **text** merupakan state. Ketika isi Text Field berubah, nilai **text** ikut berubah dan Compose melakukan recomposition, sehingga tampilan Text Field diperbarui.

18. Sebutkan state apa saja yang ada pada program yang terakhir dibuat?

Ada dua state yaitu count untuk menyimpan jumlah klik tombol Tambah dan juga text, menyimpan isi Text Field.

19. Sebutkan event apa saja yang ada pada program yang terakhir dibuat?

Ada dua event yaitu onClick, event pada tombol Tambah. Dan juga onChange, event pada Text Field.

20. Kapan event - event tersebut dijalankan?

- onClick dijalankan saat tombol Tambah diklik.
- onChange dijalankan saat isi Text Field berubah, misalnya ketika mengetik atau menghapus huruf

Implisit Intent

1. Buat sebuah Project Android Studio baru, dengan Template Empty Activity untuk Phone and Tablet. Pastikan project yang dibuat menggunakan Jetpack Compose. Tunggu sampai proses Build inisialisasi project baru ini selesai dilakukan sebelum melanjutkan percobaan ke langkah selanjutnya.
2. Buka file MainActivity.kt dan tambahkan fungsi composable berikut ini di luar class MainActivity

@Composable

```
fun ImplicitIntentTest(context: Context) {  
    Column(  
        verticalArrangement = Arrangement.Center,  
        horizontalAlignment = Alignment.CenterHorizontally,  
        modifier = Modifier.fillMaxSize()  
    ) {  
        // Buka Browser  
        Button(onClick = {  
            val intent = Intent(Intent.ACTION_VIEW).apply {  
                data = Uri.parse("https://www.google.com")  
            }  
            context.startActivity(intent)  
        }) {  
            Text("Buka Browser")  
        }  
        Spacer(modifier = Modifier.height(16.dp))  
        // Buka Dial Telepon  
        Button(onClick = {  
            val intent = Intent(Intent.ACTION_DIAL).apply {
```

```

        data = Uri.parse("tel:08123456789")
    }
    context.startActivity(intent)
}) {
    Text("Buka Telepon")
}
Spacer(modifier = Modifier.height(16.dp))
// Kirim Email (dengan chooser)
Button(onClick = {
    val intent = Intent(Intent.ACTION_SENDTO).apply {
        data = Uri.parse("mailto:test@gmail.com")
    }
    val chooser = Intent.createChooser(intent, "Pilih aplikasi
email")
    context.startActivity(chooser)
}) {
    Text("Kirim Email")
}
Spacer(modifier = Modifier.height(16.dp))
// Buka Maps
Button(onClick = {
    val intent = Intent(Intent.ACTION_VIEW).apply {
        data = Uri.parse("geo:-7.250445,112.768845")
    }
    context.startActivity(intent)
}) {
    Text("Buka Maps")
}
}
}

```

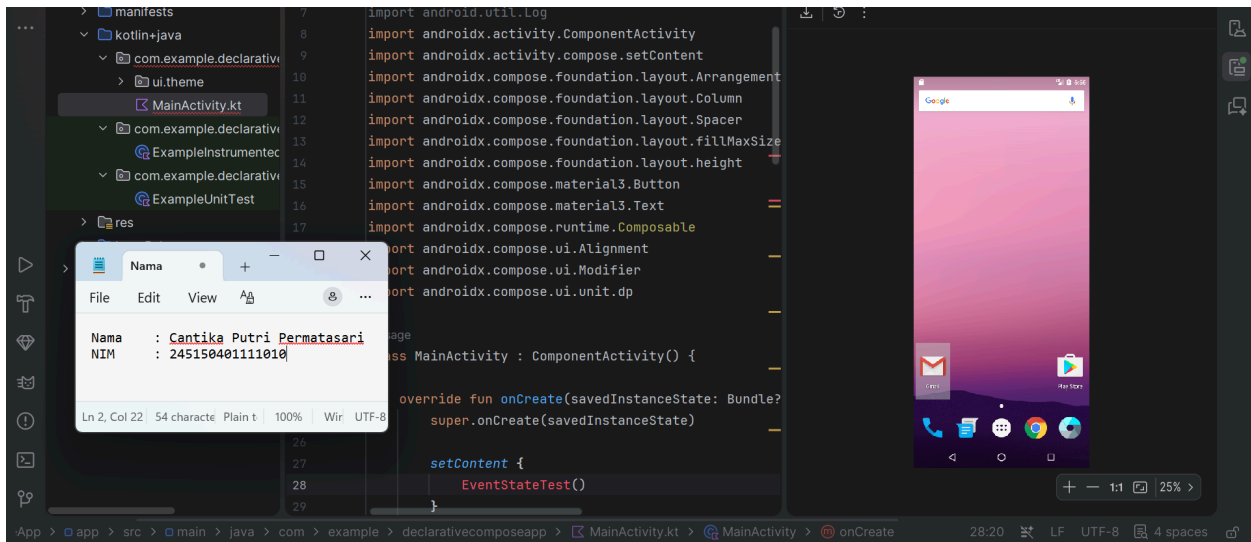
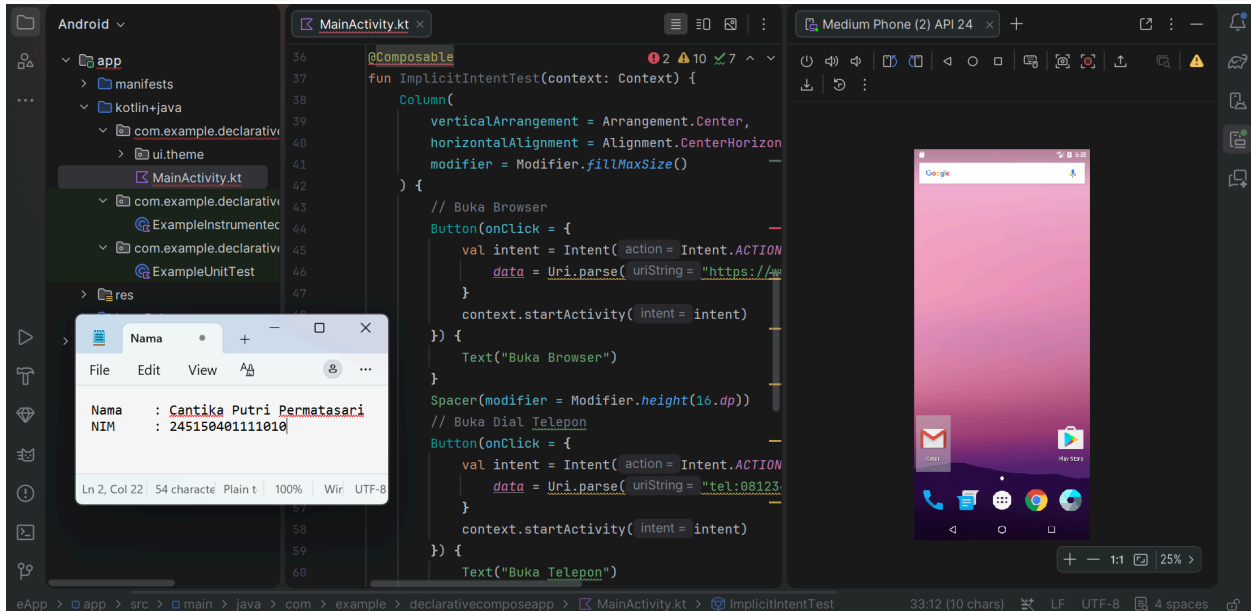
3. Import semua keyword dan konstruk atas kesalahan sintaksis dan struktur data yang muncul sebagai akibat dari penambahan kode program yang baru ditambahkan tersebut, termasuk namun tidak terbatas pada:

```

import android.content.Context
import android.content.Intent
import android.net.Uri
import androidx.compose.foundation.layout.*
import androidx.compose.material3.Button
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier

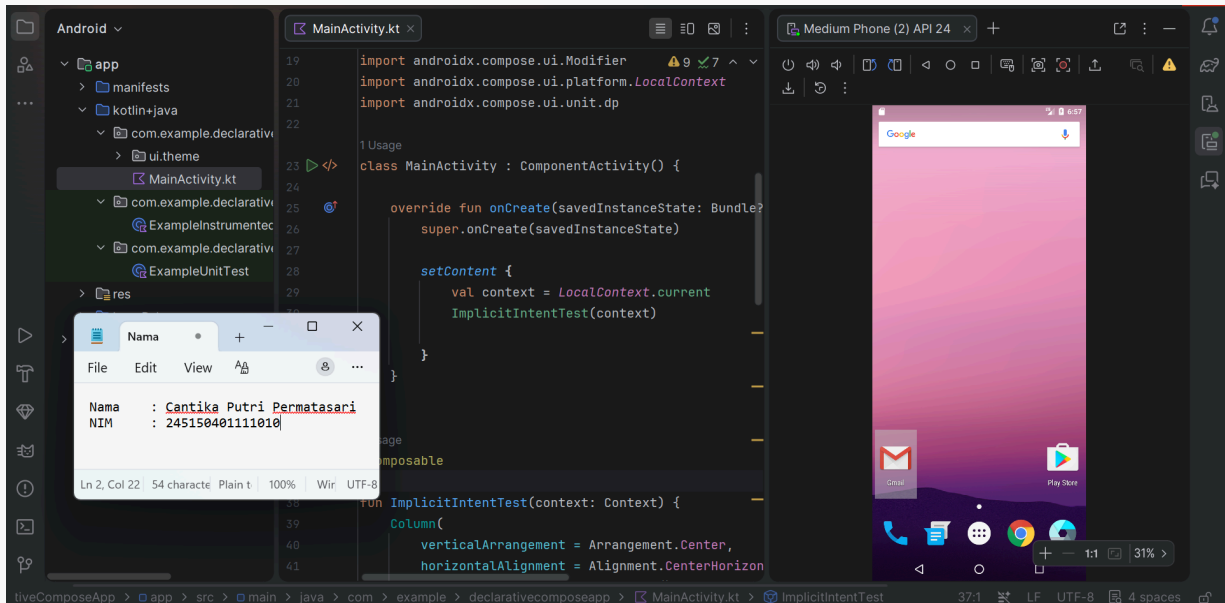
```

```
import androidx.compose.ui.unit.dp
```



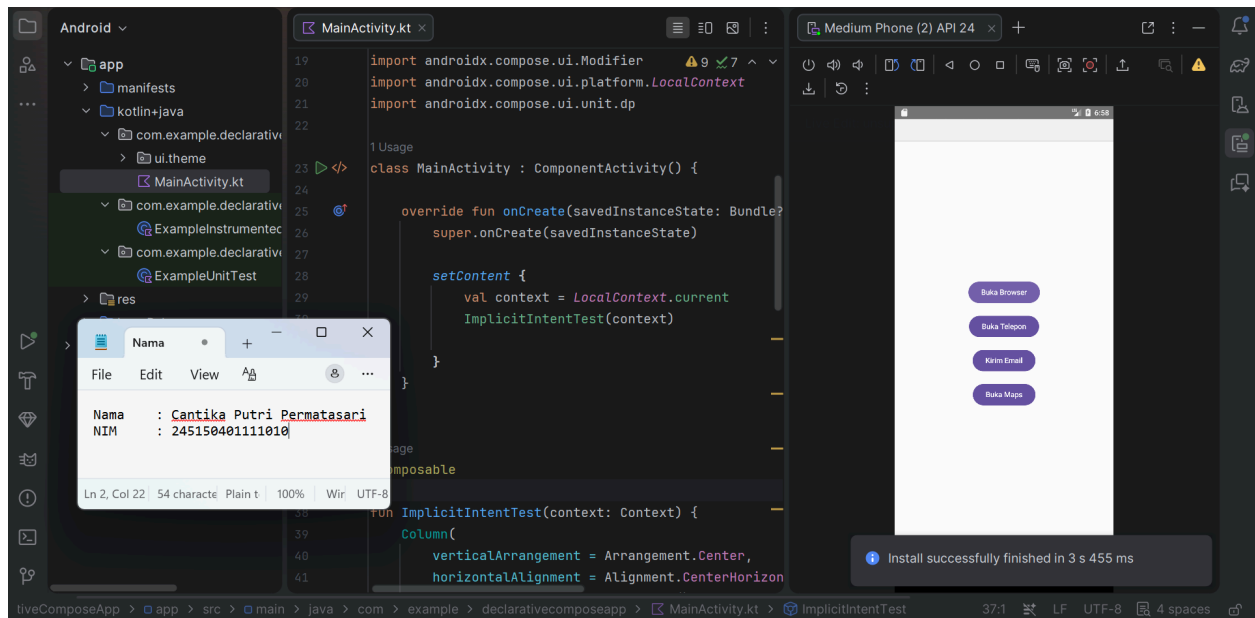
4. Buka file `MainActivity.kt`, temukan fungsi `onCreate()` yang telah disediakan oleh template aplikasi Android di Android Studio. Ubah isi `setContent { }` menjadi

```
setContent {  
    val context = LocalContext.current  
    ImplicitIntentTest(context)  
}
```

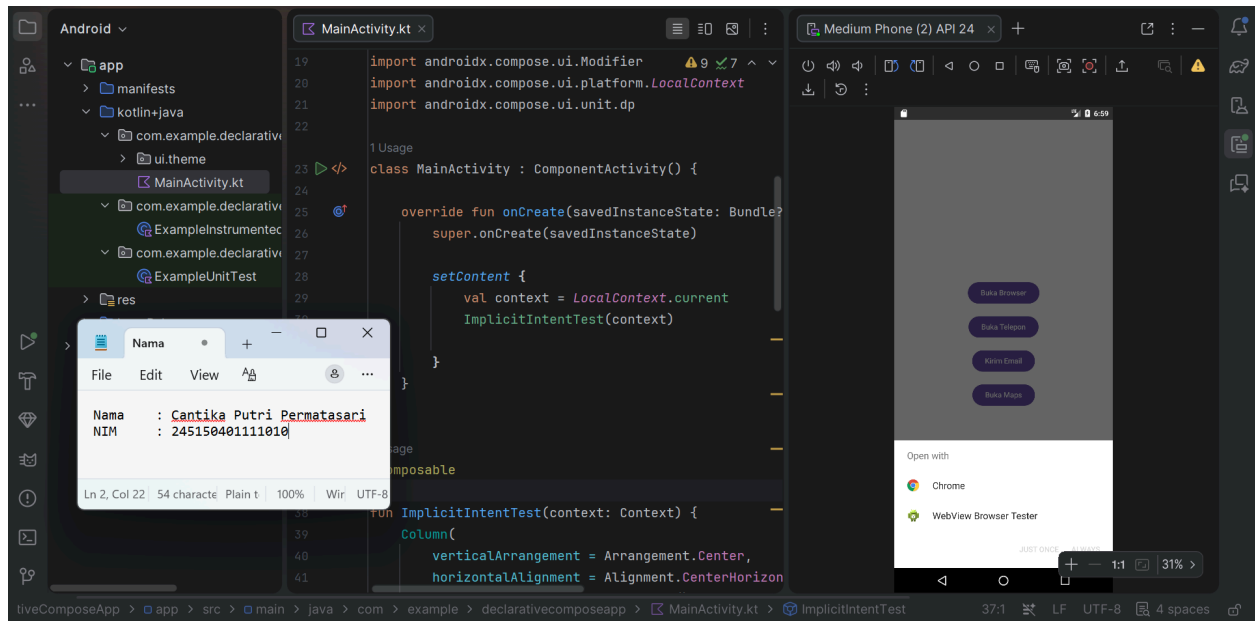


Aktivitas Percobaan

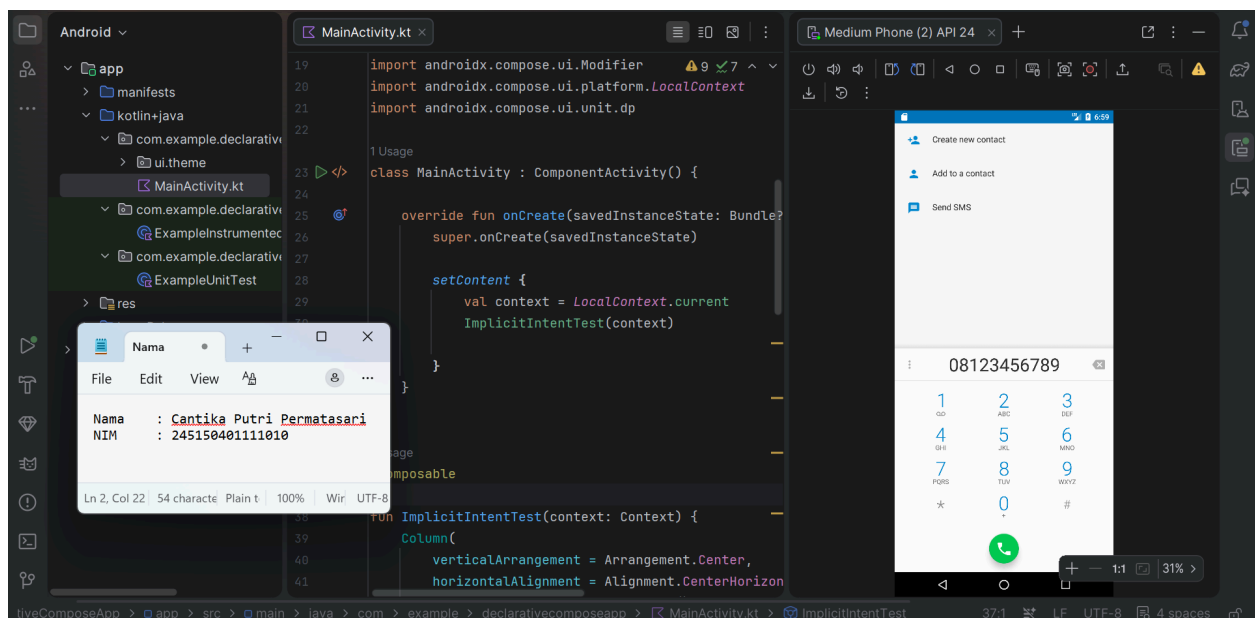
1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Lakukan pengujian pada setiap tombol:



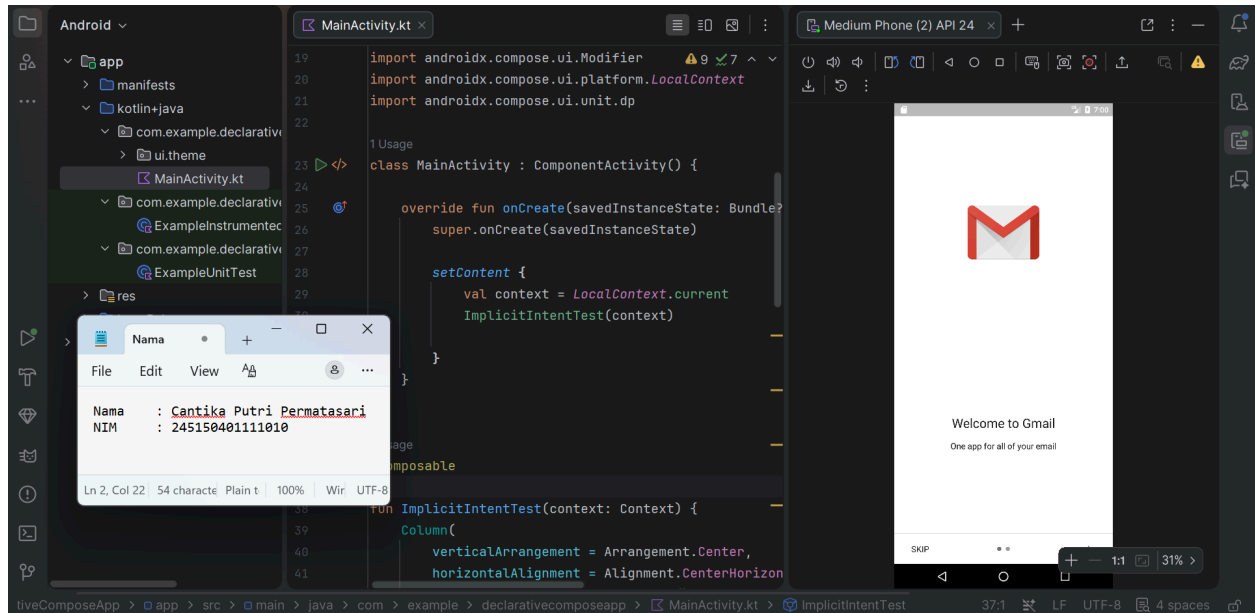
2. Klik Buka Browser



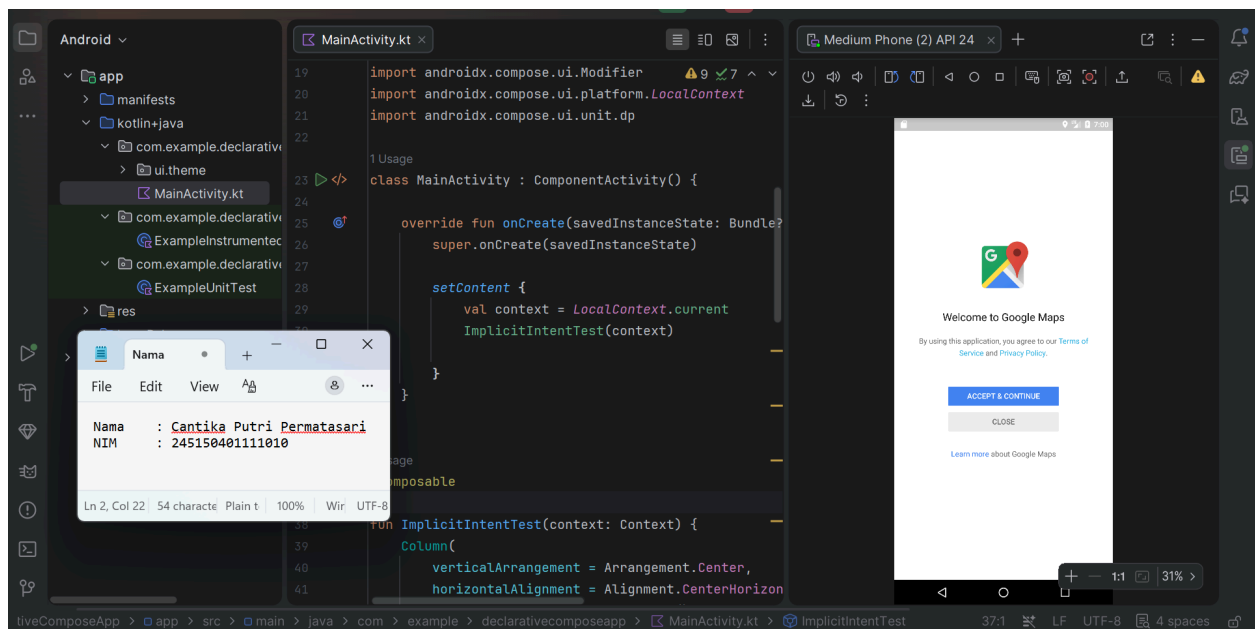
3. Klik Buka Telepon



4. Klik Kirim Email



5. Klik Buka Maps



6. Amati apa yang terjadi pada setiap aksi, kemudian ambil screenshot hasilnya

7. Pada kode yang digunakan, terdapat beberapa action seperti:

- `Intent.ACTION_VIEW`
- `Intent.ACTION_DIAL`
- `Intent.ACTION_SENDTO`

Apa perbedaan fungsi dari masing-masing action intent tersebut?

- `Intent.ACTION_VIEW`, digunakan untuk membuka atau menampilkan suatu data menggunakan aplikasi yang sesuai, seperti browser atau Maps.

- *Intent.ACTION_DIAL*, digunakan untuk membuka aplikasi telepon pada halaman dial dengan nomor yang sudah ditentukan.
 - *Intent.ACTION_SENDTO*, digunakan untuk mengirim data ke tujuan tertentu, dalam percobaan ini digunakan untuk membuka aplikasi email dengan alamat email tujuan.
8. Pada kode yang digunakan, terdapat beberapa data seperti:
- data = Uri.parse("mailto:test@gmail.com")
 - data = Uri.parse("https://www.google.com")
 - data = Uri.parse("geo:-7.250445,112.768845")

Apa fungsi data pada intent ?

Data pada intent berfungsi untuk menentukan informasi atau tujuan yang akan diproses oleh Intent

9. Apa hubungan Action dan Data pada pilihan aplikasi yang dimunculkan oleh sistem Android

Action dan Data saling berkaitan dalam menentukan aplikasi yang dapat menangani sebuah intent. Action menentukan tindakan yang ingin dilakukan, sedangkan Data menentukan objek atau informasi yang akan diproses. Sistem Android akan mencocokkan keduanya dengan aplikasi yang sesuai.